Lab #8: Design of a Traffic Controller

In this lab, you will design a traffic light controller and simulate your design. You will learn how choosing different sets of states can greatly affect the complexity of your circuit. Read the instructions carefully. Part 1 analyzes the problem and offers two solutions. Part 2 asks you to address the first solution. Part 3 guides you through the second solution. Part 4 guides you through a revised version of Part 3. **This lab is to be done with your lab partner only, and one lab report will be submitted per team. The hardware implementation is done *individually* in Lab 9.**

Part 1: Traffic Light Specifications and Design Choices

The traffic lights you wish to control are placed at an intersection of two streets. One street runs north-south (called NS) and the other street runs east-west (called EW). The red, yellow, and green of the traffic light for the street NS are denoted by R_{NS}, Y_{NS}, and G_{NS}, respectively. Similarly, the red, yellow, and green for the street EW are denoted by R_{EW}, Y_{EW}, and G_{EW}, respectively. Each light cycles through R, Y, and G repetitively, with 5 clock cycles for R, 1 clock cycle for Y, and 4 clock cycles for G. The EW light is R when the NS light is Y or G. Similarly, the NS light is R when the EW light is Y or G. Consequently, the duration of R equals the sum of the durations for Y and G (i.e., 5 = 1 + 4).

You should design the controller by noting that it is essentially a counter driven with a clock (without any user input). Since each street is R for 5 cycles, and they alternate, you will need to design the circuit to count through **10 states**.

Now, we will discuss two approaches to designing a sequential circuit that meets these specifications.

In Lab 7, each bit of your state corresponded directly with an LED signal: three state bits controlled three LEDs. It is reasonable to try to design the traffic light controller with 6 state bits, one for each LED. However, a problem is encountered: two sets of four states (those with green light NS or green light EW) have *identical LED patterns*. The *state* must change, however, to make the circuit progress after 4 cycles. If the lights are equal to the state bits, this is an obvious impossibility. To remedy this, you may add an extra 2 bits to create $2^2 = 4$ extra combinations for any of the original 6-bit states. This leads to a total of 8 state bits, wherein 6 bits describe the lights, and 2 bits allow extra states with identical lights. This is the first approach.

Another approach is to use a minimal number of state bits and design logic to convert the state bits into LED signals. Each state will have a unique pattern of 1s and 0s. Next-state

equations will determine the next-state for the counter, and each of the 6 LEDs will have its own function that takes the state as input and produces the desired light value for that LED at that state. In this lab, you will investigate both approaches.

Questions:

1. Sketch the intersection with the six traffic lamps. Label the street names and the R, G, and B lamps for each direction.
2. Using the description above, create a table that shows the state of the six lamps at each of the 10 clock cycles. Since you do not know the states yet, just label each row 1-10.

Part 2: The 8-bit Approach

Consider the design of a sequence counter (described above) that uses 6 bits that correspond directly with LED values and 2 additional bits to generate 4 extra states with identical light patterns for the green cycles. This is a total of 8 state bits.

Questions:

3. How many flip-flops (of any variety) are required to store the 8-bit state?
4. How many rows would your full transition table have?
5. How many unused states are there?
6. Imagine that when you power up your circuit, the starting state is random. What's the percentage chance that your controller *misses* your 10 used states? What dangers does this pose, and how would you try to fix it?
7. Considering the number of variables/combinations, do you think that designing an optimized circuit is practical?

Part 3: Minimal State Bits Approach, Version I

Consider the design of a sequence counter (described above) using a minimal number of state bits to describe all 10 states of the traffic controller. Remember: each LED will need an extra Boolean expression to calculate its value from the state bits. For this part, you will solve the circuit using a sequence that counts from 0 to 9. In the next part, you will change the sequence to simplify the problem.

Questions:

8. What's the minimum number of state bits required to support the 10 traffic controller states?
9. How many rows will the transition table contain?
10. This controller's sequence counter will count from 0 to 9 in binary and repeat: $ABCD = 0000 \rightarrow 0001 \rightarrow 0010 \rightarrow 0011 \rightarrow 0100 \rightarrow 0101 \rightarrow 0110 \rightarrow 0111 \rightarrow 1000 \rightarrow 1001 \rightarrow 0000$. Create a 16-row transition table. Add columns for each of the six traffic light lamps as well. Leave unused states as don't-cares.
11. Using K-maps, solve for A+, B+, C+, D+, R_{NS}, G_{NS}, Y_{NS}, R_{EW}, G_{EW}, and Y_{EW} as functions of $ABCD$ given that $Y_{NS} = 1$ when $ABCD = 0100$. Show your work and place your formulas in a table.

Part 4: Minimal State Bits Approach, Version II

That wasn't fun, was it? It would be a terrible circuit to build in Lab 9. Many people have done it like this. Fortunately, there is a better way. The complicated logical results from Part 3 can be largely eliminated by simply changing the sequence! Counting from 0 to 9 is not an optimal solution for this problem.

Consider a thought experiment. Suppose you have to make a sequence that you will explain to another person. If the sequence is random, then you must do a lot of explaining. This is because there isn't any simple reasoning that connects one state to the next. However, if you create the sequence with a recognizable pattern, then it will be very easy to explain. In the first case, your long and complicated explanation would correspond with a massive Boolean algebra expression. In the second case, your short and simple explanation would correspond with a tiny Boolean algebra expression. So, while it does seem strange, it is possible to ensure that your Boolean expressions are simple *before you even draw K-maps* just by using your creativity to invent a sequence with a simple governing pattern. Furthermore, if your sequence has some correlation to the state of the individual traffic light lamps, then the formulas that convert the state into light values will also be simple because they obey a simple explanation.

Your task is to invent a 10-cycle, 4-bit sequence that is easy to explain and is also modeled after the light patterns. This will ensure trivial Boolean algebra expressions and an easy Lab 9. This has been done with only 12 gates and flip-flops! Feel free to discuss solutions with your groupmate and other classmates. Hints are given below to suggest a sequence that correlates well with the lights.

Hint 1: Note that R_{NS} and R_{EW} are each illuminated exactly half of the time. It is worth considering the use of a single bit to symbolize the state of these two lights.

Hint 2: The green lights last 4 clock cycles. How many bits are required to make 4 combinations? Recall the Johnson counter from Lab 6.

Hint 3: The yellow lights are on briefly, and a single bit can perhaps be used to indicate that a yellow light is on.

Questions:

12. Create a 16-row transition table for your homemade states. Add additional columns for each of the six traffic light lamps. Leave the unused rows as don't-cares.
13. Explain (with words) how to find the state of G_{EW} given any state in your sequence.
14. Solve for A+, B+, C+, D+, R_{NS}, G_{NS}, Y_{NS}, R_{EW}, G_{EW}, and Y_{EW} using K-maps. Show your work.
15. Does your Boolean algebra solution for G_{EW} match your verbal explanation above?
16. Provide a circuit schematic for your solution.
17. Provide a timing diagram for ten clock cycles.
18. Was your solution better the one in Part 3? If not, try again! It will be worth it.

Part 5: Summary of deliverables

Your report should follow the format listed below and enumerate and address every numbered question. **Please label your circuit properly. Points will be deducted if the circuit is not labeled properly. Label the inputs and outputs of all flip-flops and label all streetlights appropriately.**

Turn in your report in either PDF or Word format, and make sure it is named: **ECE3441_Team#_Lab8**.

****Note: Please refrain from using notebook paper or writing on the attached lab. Make a separate Word document and have it submitted as a word or as a pdf document in BB. Points will be deducted if you do not follow instructions.**

Use the following structure for your lab report:

Course Name and Number – Semester
Lab Number and Title
Names: Lab Partner(s)

Overview: Goals of the Lab

Briefly (~ 1 small paragraph) outline the goal of this lab, as well as the work done.

Step-by-Step Procedures and Results

This must be in chronological order of how Lab 8 should be completed. Make sure you include all the necessary information that is mentioned in the lab and answer all questions.

Conclusion

Summarize what you have learned in this lab.